# Task and Motion Planning

## Contents


1. Overview . . . 2
2. Why the list of actions fails: the numbers inside each action . . . 4
   - 2.1 What the facts leave out . . . 4
   - 2.2 A skill and its numbers . . . 4
3. Searching for the actions and their numbers together . . . 6
   - 3.1 Plan skeletons, and a failed test that becomes a fact . . . 6
   - 3.2 Sampling the bottle's place, and backing up . . . 7
   - 3.3 The plan . . . 10
   - 3.4 Two orders for the two searches . . . 11
4. The code real robots use: PDDLStream . . . 12
5. A request in words: a language model chooses the skills . . . 13
   - 5.1 Say: the language model's score for each skill . . . 13
   - 5.2 Can: the chance the skill succeeds now . . . 15
   - 5.3 Say times Can, one skill at a time . . . 15
   - 5.4 SayCan on a real robot, and what each method needs . . . 17
6. Summary . . . 18
7. Sources . . . 19
8. Key terms . . . 20

**Where this fits:**

- **Builds on:** Reinforcement learning; Large language models (LLMs); PDDL; STRIPS actions; Manipulation planning (transit and transfer).

## 1. Overview

**Key point:** A list of actions found from facts alone can ask the arm for something the table does not allow, such as putting the cup down where a bottle stands. Task and motion planning searches for the actions and for their numbers (places, joint angles, motions) together, and a language model can choose skills from a request in words if each skill's score is multiplied by the chance the robot can carry it out now.

The robot can already do three things. Its task planner puts actions in order from true-or-false facts, such as "pick up the cup, then put it down at the door". Its arm can plan the transit and transfer motions that carry a cup from spot A to spot B on a table. And Nav2 (G-3340), the navigation stack of this chapter, drives its base to any pose in the room. The three are planned separately, and the task planner never looks at the table.

So we return to the table top of manipulation planning, where the actions meet the geometry: the arm with links of 0.3 m and 0.2 m, its shoulder 0.1 m above the table, the cup (6 cm wide, 8 cm tall) at A, 0.18 m from the shoulder. The bottle (4 cm wide, 14 cm tall) has been left standing on spot B, 0.42 m out. The request: put the cup on spot B.

The facts say the cup is at A and the hand is empty. From these facts the task planner returns two actions, and Figure 1 checks them against the table:

1. pick up the cup at A: the arm finds a motion to the cup's lid, 154.6 degrees of joint motion;
2. put the cup down at B: the cup at B would overlap the bottle by 4 cm, the bottle's whole width, so no placement exists.

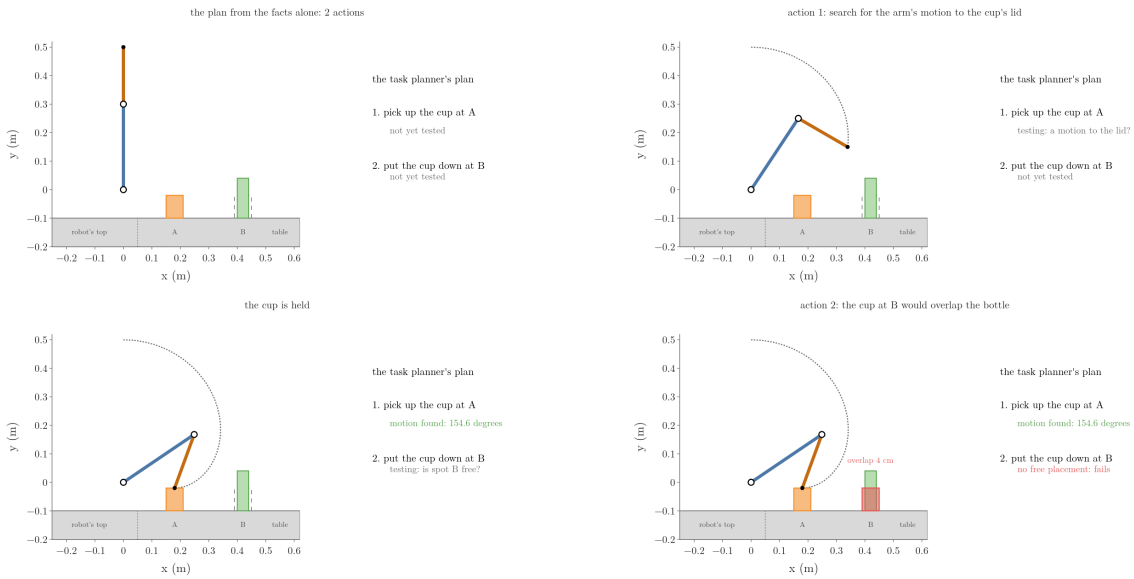



**Figure 1:** Animation: the task planner's two-action plan checked against the table by our Python planner (`tampkit.py`, not PDDLStream), side view. The arm (blue and brown links) starts straight up; the cup (orange) stands at A, the bottle (green) on spot B (dashed outline). Action 1, pick up the cup at A: the planner searches for the arm's motion to the cup's lid and finds one, 154.6 degrees of joint motion (the dotted line is the hand's track). Action 2, put the cup down at B: the cup drawn at B (red) would overlap the bottle by 4 cm, so the plan fails

The plan fails at its second action, after the first motion has already been planned, because nothing in the facts states that the bottle stands on B. There is a second problem: a person does not give the robot facts, but a request in words, "put the cup on spot B".

The opening question of this Note: **how does the robot find a list of actions that the arm can carry out on this table, and how does it choose those actions from a request in words?** We answer it in steps:

- why the list of actions fails, and what numbers each action hides (Section 2);
- searching for the actions and their numbers together on the cup and the bottle (Section 3);
- the code base that does this on real robots, PDDLStream (Section 4);
- a language model choosing skills from the request, the way SayCan does (Section 5).

Every run in this Note is real code in the notebook `RO-081-task-and-motion-planning.ipynb`: the planner in `images/tampkit.py` (extended from the arm model of manipulation planning) and GPT-2 small for the language scores. Every number comes from its output.

## 2. Why the list of actions fails: the numbers inside each action

> **Key point:** The two-action plan of the Overview fails because "put the cup down at B" hides numbers: where the cup will stand, the joint angles that put the hand there, and the motion. Each action is a skill whose numbers must pass tests (on the table, free of other objects, within the arm's reach, a free motion) before the action can run.

### 2.1 What the facts leave out

The Overview's plan broke at "put the cup down at B" with a 4 cm overlap. The facts the task planner works from are STRIPS-style facts (G-2601), true-or-false statements such as `cup-at A` and `hand-empty`. The precondition (G-2606) of a put-down is `holding cup`, and that fact holds after the pick-up, so the task planner sees nothing wrong.

What decides whether the put-down can run is geometry: the cup, 6 cm wide, needs 6 cm of table at B, and the bottle fills 4 cm of it. The facts carry no widths and no positions. So the task planner assumes that every list of actions it finds can be carried out by the arm. The first planners, on the Shakey robot, made the same assumption (Garrett §3.3.1); on a table with objects it rarely holds, because whether one action can run depends on where earlier actions left things (Garrett §1.1; Robohub TAMP).

A motion planner alone cannot solve this either. A motion planner moves the arm from one configuration to another but never changes where the objects stand; the mode (G-2803), which objects stand where and what the hand holds, changes only at a grasp or a release (Garrett §2.2). Moving the bottle off B is a change of mode, and choosing to make it is a decision about actions. So the robot needs both searches at once: one over actions, one over the numbers inside them.

### 2.2 A skill and its numbers

Each action the robot can carry out on command, such as "pick up the bottle", with the preconditions it needs and a motion that carries it out, is a **skill** (G-3331) (Ahn et al. §3). A pick-up skill hides four numbers. Figure 2 marks them for "pick up the bottle" on our table:

- $x$ is where the object stands, the distance of its centre from the shoulder along the table: 0.42 m for the bottle;
- the grasp is where the hand holds it: the suction gripper holds a flat top, so the one grasp is the centre of the bottle's cap, at (0.42, 0.04) m;
- $q$ is the arm's configuration, its two joint angles, that puts the hand on the grasp: (31.2, −66.4) degrees;
- the motion is a free path of the arm from where it is to $q$: here 88.7 degrees of joint motion from home, found by the probabilistic roadmap (G-2780).

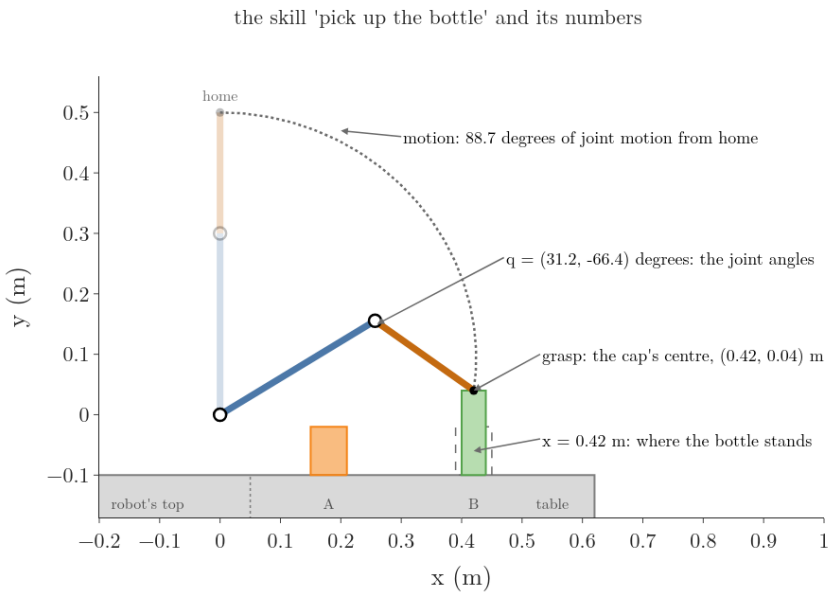



**Figure 2:** The skill "pick up the bottle" and its numbers from our Python planner, side view: the bottle (green) stands on spot B, x = 0.42 m from the shoulder; the grasp is the centre of its cap, (0.42, 0.04) m; the joint angles q = (31.2, −66.4) degrees put the hand there (the arm in blue and brown); the dotted line is the hand's track along the motion from home (the faint upright arm), 88.7 degrees of joint motion

Written as an action with parameters (G-2614), the pick-up gets these numbers as parameters, and its precondition gains tests on them. PDDLStream's example domain for a table-top arm writes it this way, shortened to the lines used here (PDDLStream `continuous_tamp/domain.pddl`):

```
(:action pick
  :parameters (?r ?b ?p ?g ?q)
  :precondition (and (Kin ?b ?q ?p ?g)
                     (AtConf ?r ?q) (AtPose ?b ?p) (HandEmpty ?r))
  :effect (and (AtGrasp ?r ?b ?g) (not (AtPose ?b ?p))
               (not (HandEmpty ?r))))
```

Reading it line by line:

- `?b` is the object, `?p` its place, `?g` the grasp, `?q` the configuration and `?r` the robot;
- `(Kin ?b ?q ?p ?g)` is the new test: with the arm at `?q`, the hand is on grasp `?g` of object `?b` standing at `?p`;
- `(AtPose ?b ?p)` and `(HandEmpty ?r)` are the facts of STRIPS, as before.

The put-down skill has the same numbers and one more test: the object at its new place must not overlap any object standing there (PDDLStream's `CFree`). The Overview's plan failed this test at B. The tests of our planner, one per kind of number:

1. **on the table:** the object's base lies on the table top, from 0.05 m (where the robot's

top ends) to 0.60 m;
2. **no collision:** the object does not overlap another standing object, a yes-or-no collision detection (G-2665);
3. **within reach:** some joint angles put the hand on its grasp without the arm hitting anything;
4. **motion:** a free path of the arm exists to those joint angles.

Finding a list of actions together with numbers for every action that pass every test is **task and motion planning** (G-3330), TAMP: task planning whose actions carry continuous parameters and constraints on them (Garrett §1, §3.1). The next section runs it on the cup and the bottle.

## 3. Searching for the actions and their numbers together

> **Key point:** Our planner first finds a list of actions from the facts, then searches for numbers that pass every test; a failed test at a named place becomes a new fact, and a failed test of a sampled number sends the search back to sample again. On the table, the second list of actions moves the bottle to 0.485 m first, after five samples, and the cup reaches B.

### 3.1 Plan skeletons, and a failed test that becomes a fact

The Overview's two actions came from the facts alone, with their numbers left open. A list of actions whose numbers are still to be found is a **plan skeleton** (G-3332) (Garrett §1.1; Robohub TAMP). Our planner searches in two loops (Garrett §3.3.1):

1. the task planner, forward search (G-2617) on the facts, returns the shortest skeleton it has not yet tried;
2. a second search tries to find numbers for every action of that skeleton that pass every test;
3. if it succeeds, the skeleton with its numbers is the plan; if not, the search returns to step 1.

Ordering the search this way, the whole skeleton first and its numbers after, is called **sequence before satisfy** (G-3335) (Garrett §3.3.1).

For the task planner, a place is a name: A and B for the cup, "start" for where the bottle stands, and "new" for a place on the table whose position the second search will choose. The first skeleton is the Overview's: pick up the cup at A, put it down at B. Its test at B fails, and the planner turns the failure into a new fact the task planner must respect: the cup at B collides with the bottle where the bottle stands. Recording a failed test as a counterexample, so that every later skeleton with the same failure is skipped, is what the survey calls a negative method (Garrett §3.4.2).

With that fact, putting the cup at B requires the bottle to move first, and the shortest skeleton becomes four actions. A person setting a table follows the same order: when a bottle stands where the cup belongs, the bottle is moved first.

1. pick up the bottle;
2. put the bottle down at a new place;
3. pick up the cup at A;
4. put the cup down at B.

The second skeleton has one number that is still open: where the bottle goes.

### 3.2 Sampling the bottle's place, and backing up

The table offers infinitely many places for the bottle, so the planner cannot list them. The planner draws them at random, the way the probabilistic roadmap draws configurations: a function returns a random $x$ at which the bottle's whole base is on the table, from 0.07 m to 0.58 m (Robohub TAMP; PDDLStream `s-region`). Each sample then goes through the four tests of Section 2.2, action by action.

Figure 3 runs the whole search, one test per frame, with seed 0 as in the notebook. Watch the list on the right: each action turns green when its tests pass and red when one fails, and the samples of $x$ collect at the bottom.

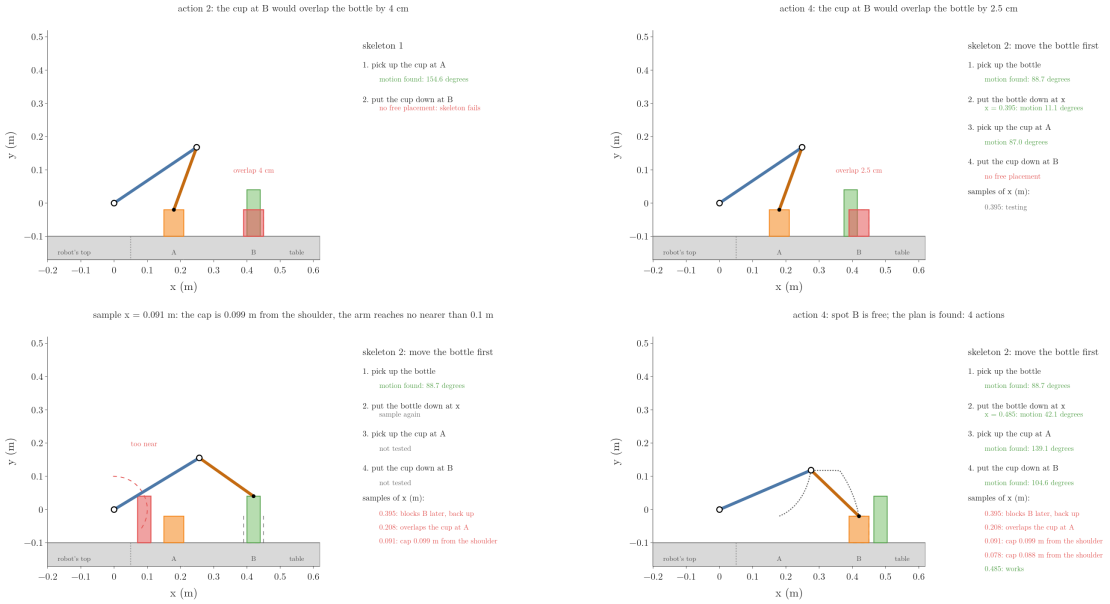



**Figure 3:** Animation: our Python planner's search (`tampkit.py`, not PDDLStream), one test per frame (seed 0). Skeleton 1: the motion to the cup's lid is found (154.6 degrees); the cup at B would overlap the bottle by 4 cm, so the skeleton fails. Frame 'new fact': the cup at B collides with the bottle where it stands; skeleton 2 moves the bottle first. Action 1, pick up the bottle: 88.7 degrees. Sample x = 0.395 m (the bottle's distance from the shoulder): free, reachable, motion 11.1 degrees; action 3, pick up the cup: 87.0 degrees; action 4: the cup at B would overlap the bottle by 2.5 cm. Frame 'backtrack': the last two motions are dropped. Sample 0.208 m: the bottle would overlap the cup at A by 2.2 cm. Samples 0.091 m and 0.078 m: the cap is 0.099 m and 0.088 m from the shoulder, nearer than the arm reaches (the red dashed circle, 0.1 m). Sample 0.485 m: motion 42.1 degrees; pick up the cup, 139.1 degrees; put it down at B, 104.6 degrees: the plan is found, 4 actions

**The collision test.** Each object covers half its width on each side of its centre, so two objects reach towards each other by half their widths added. If their centres are at least that far apart, a gap is left; if closer, they overlap. With:

- $x_1$ and $x_2$ are the two centres, in m;
- $|x_1 - x_2|$ is the gap between the centres, their difference taken as positive whichever is larger, in m;
- $w_1$ and $w_2$ are the two widths: 0.06 m for the cup, 0.04 m for the bottle.

The two are free of each other when:

$$|x_1 - x_2| \geq \frac{w_1 + w_2}{2}$$

For the cup and the bottle, half the widths added:

$$\frac{0.06 + 0.04}{2} = 0.05 \text{ m}$$

The first sample puts the bottle at 0.3949 m, and action 4 then tests the cup at B, 0.42 m. The gap between the centres:

$$0.42 - 0.3949 = 0.0251 \text{ m}$$

$$0.0251 < 0.05$$

So the two overlap, from the cup's left end, 0.39 m, to the bottle's right end, 0.4149 m:

$$0.4149 - 0.39 = 0.0249 \text{ m}$$

In the opening, both centres were at 0.42 m, a gap of 0, and the bottle lay wholly inside the cup's 6 cm: the overlap was the bottle's whole width, 4 cm.

**Backing up.** The first sample passed every test of its own action and of action 3, so the planner had already planned two motions, 11.1 and 87.0 degrees, before action 4 failed. Only the bottle's place can be changed, so the search goes back to that choice, drops the two motions and samples again. Going back to the latest choice and trying another value when a later test fails is **backtracking** (G-3334) (Robohub TAMP; Garrett §3.3.1). The 2.5 cm overlap involves a sampled place, not two named ones, so the planner samples again instead of turning it into a new fact.

**The next three samples** fail early:

- 0.2076 m: its centre is only 0.0276 m from the cup's at A, less than 0.05 m, so the bottle would overlap the cup by 0.0224 m (collision test);
- 0.0909 m and 0.0784 m: the hand cannot reach the cap (within-reach test).

**The within-reach test.** Folded fully, the arm's hand is no nearer to the shoulder than the difference of its links; stretched fully, no farther than their sum (the triangle of the two links of manipulation planning). With:

- $L_1$ and $L_2$ are the link lengths, 0.3 m and 0.2 m;
- $(x, y)$ is the cap's position from the shoulder, along the table and up, in m: (0.0909, 0.04) for the third sample, because the cap stands 0.04 m above the shoulder's height;
- $d$ is the cap's distance from the shoulder, in m.

The nearest and farthest the hand reaches:

$$0.3 - 0.2 = 0.1 \text{ m}$$

$$0.3 + 0.2 = 0.5 \text{ m}$$

The third sample's cap:

$$d = \sqrt{0.0909^2 + 0.04^2}$$

$$= \sqrt{0.008263 + 0.0016}$$

$$= 0.0993 \text{ m}$$

which is less than 0.1 m, so no joint angles put the hand there. At 0.0784 m the distance is 0.0880 m. The same test in symbols:

$$d = \sqrt{x^2 + y^2}$$

$$L_1 - L_2 \leq d \leq L_1 + L_2$$

**The fifth sample**, 0.4848 m, passes everything. The bottle there spans 0.4648 m to 0.5048 m and leaves 1.48 cm between itself and the cup at B (no collision), its cap is within reach, and a motion of 42.1 degrees takes it there. The cup's pick-up and put-down then pass too, and the search ends.

So the bottle's place is found by drawing values and testing them, and the one value that failed only at a later action, 0.395 m, cost two motions that were discarded. The next section runs the plan it found.

### 3.3 The plan

Section 3.2 ended with numbers for all four actions. Figure 4 runs them, one skill (G-3331), an action with its numbers, after another. Watch the hand's dotted track in each skill: the cup is carried over the bottle's new place only after the bottle has left B.

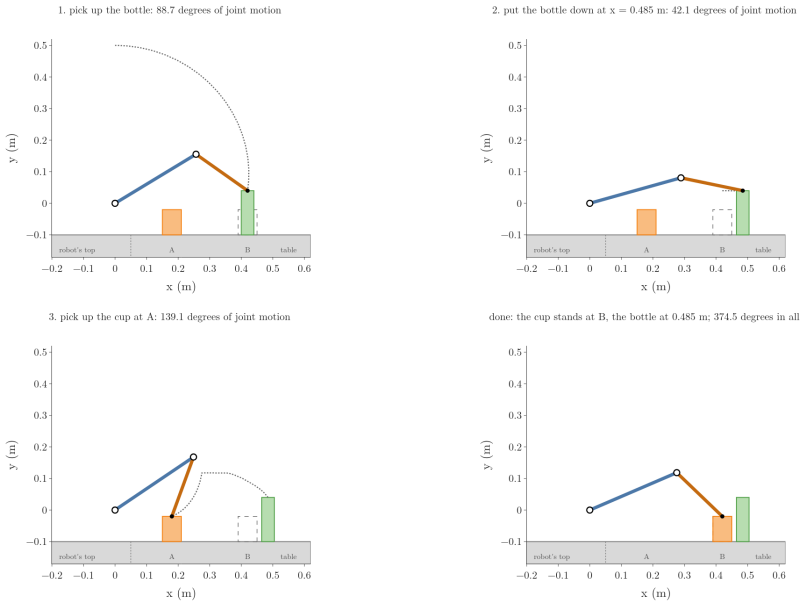



**Figure 4:** Animation: the plan found by our Python planner (`tampkit.py`, not PDDLStream), run skill by skill. 1. pick up the bottle: 88.7 degrees of joint motion; 2. put the bottle down at x = 0.485 m from the shoulder: 42.1 degrees; 3. pick up the cup at A: 139.1 degrees; 4. put the cup down at B: 104.6 degrees. The dotted line is the hand's track in the current skill. Last frame: the cup stands at B, the bottle at 0.485 m; 374.5 degrees in all

The four skills and their numbers (notebook):

| Skill | Place $x$ (m) | Joint angles $q$ (degrees) | Motion (degrees) |
|---|---|---|---|
| pick up the bottle | 0.42 | (31.2, −66.4) | 88.7 |
| put the bottle down | 0.4848 | (15.6, −27.3) | 42.1 |
| pick up the cup | 0.18 | (34.0, −144.1) | 139.1 |
| put the cup down | 0.42 | (23.2, −67.0) | 104.6 |

In the table, $x$ is where the object stands and $q$ the two joint angles at the end of the skill. The motions add up:

$$88.7 + 42.1 + 139.1 + 104.6$$

$$= 374.5 \text{ degrees}$$

The cup's two configurations are the grasp and the placing of manipulation planning, because the cup, A and B are the same. So the opening question has its first answer: four actions, with the bottle moved first to 0.485 m. To get there the planner tried 2 skeletons, drew 5 places and planned 7 motions, of which 3 were thrown away (154.6 degrees in skeleton 1, 11.1 and 87.0 degrees for the first sample). Section 3.4 asks whether the other order of the two searches would waste less.

### 3.4 Two orders for the two searches

Our planner chose the whole skeleton first and its numbers after, and threw away 3 of its 7 motions on the way. The other order is possible too, and the survey groups planners by it (Garrett §3.3). Figure 5 lights both as loops: the top one along our run, the bottom one as a schematic. Watch where each loop goes back to: to a new skeleton when a named test fails, or to more samples when no plan is found.




**Figure 5:** Animation: two ways to combine the search over actions with the search over numbers, one box lit per frame. Top, sequence before satisfy, lit along our run: skeleton 1; its numbers fail at B; frame 'no: the cup at B collides with the bottle where it stands, a new fact' (red loop back); skeleton 2; 5 samples of the bottle's place with one backtrack; every test passes; the plan, 4 actions and 374.5 degrees. Bottom, satisfy before sequence, PDDLStream's incremental algorithm as a schematic (not run in the notebook): the samplers give values; the task planner searches with tested values only; frame 'no plan yet: sample more values' (red loop back); more values; search again; a plan is found

The two orders:

- sequence before satisfy (G-3335) finds a whole skeleton, then its numbers; it wastes no samples on actions no plan needs, but a skeleton can fail late, as action 4 did for the sample 0.395 m;

- **satisfy before sequence** (G-3336) first samples values and tests them one constraint at a time (places on the table, joint angles for each place, motions between joint angles), then lets the task planner search for a plan that uses only values already tested; if none exists, it samples more and searches again (Garrett §3.3.2).

Most planners in the survey mix the two, adding actions and solving their numbers as they go (Garrett §3.3, Table 1). In each order the loop back to the start is backtracking in another form, so no order avoids it (Robohub TAMP). PDDLStream, the code base of the next section, offers both orders.

## 4. The code real robots use: PDDLStream

> **Key point:** Our planner was written for one table; PDDLStream writes the same samplers and tests as streams, Python functions next to a PDDL domain, and alternates between sampling with them and searching for actions with a PDDL planner. Planning takes seconds to minutes, and the plan is not changed while it runs.

Section 3 wrote its own tests and samplers. An open-source planner by the survey's first author writes the same pieces in files a general planner can read. **PDDLStream** (G-3337) is a planning framework that extends PDDL (G-2612) with streams (PDDLStream README). A **stream** (G-3333) is a user-written Python function that takes values and returns new values that satisfy a condition, such as a place on the table, or that only checks whether given values satisfy one; each value it returns comes with the facts it certifies, which the actions then use as preconditions (PDDLStream README; Robohub TAMP).

The example `examples/continuous_tamp` in the repository (`caelan/pddlstream`, branch `main`, read 2026-10-12) is close to our table: blocks stand on a line of ground, drawn from the side, and a suction gripper picks them from above; its collision test is the overlap of two blocks' extents along the ground, as ours is (`primitives.py`, `collision_test`). Its file `stream.pddl` declares the streams our planner wrote as Python functions:

| Stream in `stream.pddl` | What it returns | In our planner (`tampkit.py`) |
|---|---|---|
| `s-region` | a place `?p` for block `?b` inside region `?r`; certifies `Contain`, `Pose` | `sample_place`: a random $x$ on the table |
| `s-grasp` | a grasp `?g` for block `?b`; certifies `Grasp` | the one grasp, the centre of the top |
| `s-ik` | a configuration `?q` that puts the hand on grasp `?g` of `?b` at `?p`; certifies `Kin` | `grasp_conf`: the within-reach test |
| `s-motion` | a path `?t` from `?q1` to `?q2`; certifies `Motion` | `motion`: the probabilistic roadmap |
| `t-cfree` | nothing: a test; certifies `CFree` when two placed blocks do not overlap | `cfree`: the collision test |

The `place` action of `domain.pddl` asks `CFree` for the new place against every block standing anywhere, the test the Overview's plan failed. PDDLStream has several algorithms on the same files (README, "Algorithms"):

- **Incremental** samples values with the streams, hands the facts they certify to a PDDL planner, Fast Downward, and samples more when no plan is found: satisfy before sequence, much as a probabilistic roadmap samples configurations and then searches the graph they make;
- **Focused**, **Binding** and **Adaptive** plan first with placeholder values for numbers not yet sampled, then call only the streams that plan needs (Garrett §3.4.1); the README says Adaptive "typically performs best for domains with many possible sampling pathways, such as robot manipulation domains".

The open-source robot simulator `pyrobosim` uses PDDLStream for its task and motion planning (README, "Applications"; Robohub TAMP walks through it). Two limits come with the method:

- planning a task such as twisting the cap off a bottle took from under a minute to two or three minutes, much of it spent sampling (sampling can need a very long time when few values satisfy a constraint, Garrett §3.3);
- the plan is computed before the robot moves and is not replanned if something changes while it runs.

On a robot, the skills (G-3331) of such a plan are run by the behaviour layer of the autonomy stack (G-3210), the robot's software arranged in layers from mission down to control. A skill that moves the base, such as "go to the table", becomes a goal for a classical navigation stack, as in SayCan (Ahn et al. App. D.2) and in Nav2 (G-3340). What PDDLStream still needs is a goal written as facts, and Section 5 starts from a request in words.

## 5. A request in words: a language model chooses the skills

> **Key point:** For the request in words, SayCan multiplies, for each skill, the language model's probability that the skill's words come next (Say) by the chance the skill succeeds now (Can), and runs the best product, one skill at a time. On our table, with GPT-2 small, Can rules out the two skills with the highest Say scores at step 2, and "done" then wins by 1.104 to 1, so the robot ends the plan with the cup still in its hand.

### 5.1 Say: the language model's score for each skill

The second problem of the Overview was the request "put the cup on spot B", given in words. A large language model (G-1046), a transformer trained on a vast amount of text, has read many descriptions of how to put things in places. Asked directly, however, it may answer with steps for which the robot has no skill: asked to help with a spill, one model suggested "You could try using a vacuum cleaner" (Ahn et al. §1). So SayCan does not let the model write freely. SayCan asks the model only to score the robot's own skills (G-3331), the actions it can carry out on command, each written in a few words such as "pick up the cup" (Ahn et al. §3).

The score is the probability that the model gives to the skill's words as the next step. A GPT (G-855) model gives a probability for every possible next token, so the probability of several tokens is the product of each token's probability given the ones before it (Ahn et al. §2). Adding their logarithms gives the same ranking with easier numbers, the log-likelihood (G-1113) of the skill's words.

We score our seven skills with GPT-2 small, a GPT (G-855) of 124 million parameters, run in the notebook. The text before the skills follows SayCan's prompt (Ahn et al. App. D.3, Listing 1): a line saying what the robot is, worked examples with other objects, then the request (the full prompt is in the notebook). For example:

```
Human: How would you put the book on the shelf?
Robot: 1. pick up the book, 2. put down the book on the shelf, 3. done.
Human: How would you put the cup on spot B?
Robot: 1.
```

At step 1, GPT-2 small gives "pick up the cup" the log-likelihood $-12.68$, a probability:

$$e^{-12.68} = 3.113 \times 10^{-6}$$

The seven skills' probabilities add up to $3.130 \times 10^{-6}$ (notebook), so the share of the seven that goes to "pick up the cup" is:

$$\frac{3.113}{3.130} = 0.9946$$

With:

- $\ell$ is a skill's words, such as "pick up the cup";
- $i$ is the request and the steps chosen so far;
- $p(\ell \mid i)$ is the language model's probability for $\ell$ after $i$, shared out over the robot's skills: the Say score.

The share is each probability divided by the sum of all seven:

$$p(\ell \mid i) = \frac{p_\ell}{\sum_k p_k}$$

where $p_\ell$ is the probability of skill $\ell$'s words and the sum runs over the seven skills $k$. Sharing out does not change which skill is best; it only makes the scores add up to 1.

The Say score is computed from words alone, not from the table. At step 1 it puts 0.0007 on "put down the cup on spot B", a skill that cannot run with an empty hand. A second score must state what the robot can do now.

### 5.2 Can: the chance the skill succeeds now

SayCan gives every skill a second number: the probability that the skill, started in the present state, succeeds. This probability is the skill's **affordance** (G-3339): what the scene allows the robot to do right now (Ahn et al. §3). In SayCan each skill's affordance comes from a different source (Ahn et al. App. D.2):

- pick-up skills: an estimate of success learned by trial and error, by reinforcement learning (G-1660), rescaled to 0 to 1;
- go-to skills: from the distance to the place; the skill itself drives the base with a classical navigation stack;
- place skills: always 1;
- "done": a small fixed value, 0.1, so the robot stops when no skill is possible.

Our affordance uses the four tests of Section 2.2, which our planner already computes: 1 when the skill's place, joint angles and motion pass every test in the present state, 0 when one fails, and SayCan's 0.1 for "done". "Put down the cup on the table" means a free place anywhere on the table outside spot B, drawn by the sampler of Section 3.2. With:

- $s$ is the present state: where the objects stand and what the hand holds;
- $p(c \mid s, \ell)$ is the probability that skill $\ell$ completes from $s$: the Can score.

Each skill now has two numbers, Say from the words and Can from the table; Section 5.3 combines them into one.

### 5.3 Say times Can, one skill at a time

SayCan needs one number per skill: the chance that running the skill now moves the request forward. A skill moves the request forward only if it succeeds and is the right next step. If it fails, it moves nothing (Ahn et al. §3). For "done" at step 2:

$$0.1 \times 0.01735 + 0.9 \times 0$$

$$= 0.001735$$

In symbols, the chance that the skill completes times the chance it is the right step, plus the chance it fails times zero:

$$p(c \mid s, \ell)\, p(\ell \mid i) + (1 - p(c \mid s, \ell)) \times 0$$

$$= p(c \mid s, \ell)\, p(\ell \mid i)$$

So the score of a skill is Say times Can, and SayCan runs the skill with the largest product, adds its words to the text, and scores again, until "done" wins (Ahn et al. Algorithm 1).

Choosing skills by Say times Can is **SayCan** (G-3338), a method from Robotics at Google and Everyday Robots named after its two scores (Ahn et al. §1).

Figure 6 runs it on our table from the notebook's numbers. Watch the green column, Say times Can, shared out over the skills: at step 1 the skill with the highest Say score is possible and is chosen; at step 2 the two skills with the highest Say scores get Can 0.

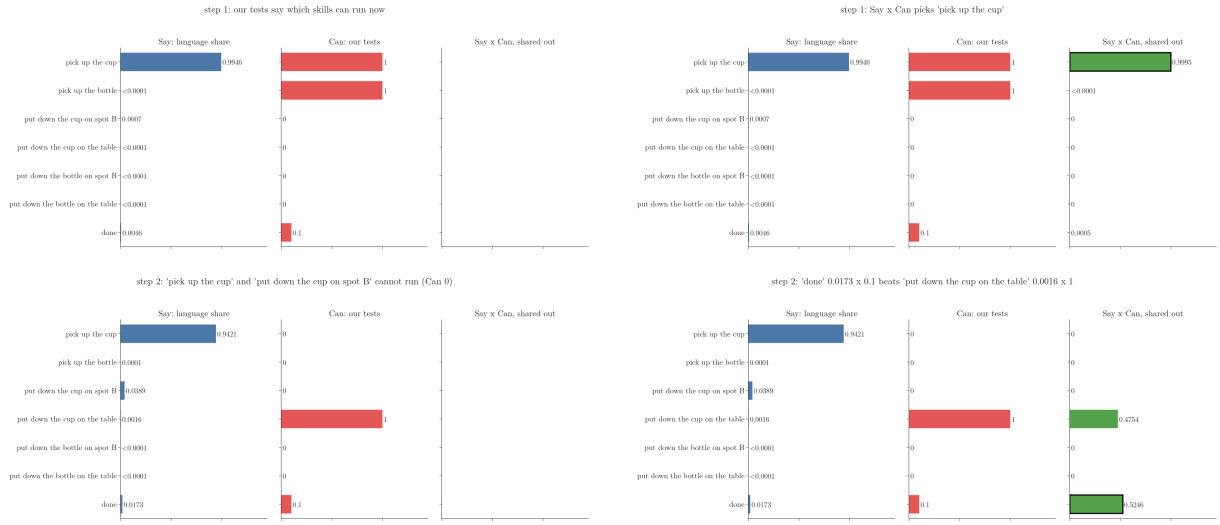



**Figure 6:** Animation: SayCan's choice on the table, one skill at a time, in our run with GPT-2 small (SayCan itself uses PaLM). Rows: the seven skills. Blue: Say, the language model's share for each skill; red: Can, our tests in the present state (1 possible, 0 not, 0.1 for done); green: Say times Can, shared out, with the chosen skill outlined. Step 1, the hand is empty: 'pick up the cup' gets Say 0.9946 and Can 1 and is chosen. Step 2, the cup is held: 'pick up the cup' (Say 0.9421) and 'put down the cup on spot B' (Say 0.0389) get Can 0; 'done' 0.0173 times 0.1 beats 'put down the cup on the table' 0.0016 times 1, shared out 0.5246 to 0.4754. Last frame: SayCan's plan is 1. pick up the cup, 2. done; the cup stays in the hand

At step 2 the hand holds the cup, and the products of the two possible skills are:

$$\text{done: } 0.01735 \times 0.1 = 0.001735$$

$$\text{table: } 0.001572 \times 1 = 0.001572$$

$$0.001735/0.001572 = 1.104$$

GPT-2 small gives "done" 11.04 times the Say score of "put down the cup on the table" (notebook); Can multiplies the score of "done" by 0.1, which leaves "done" 1.104 times ahead.

So SayCan's plan is "pick up the cup, done", and the robot ends the plan with the cup in its hand, short of B.

The Can score removed "pick up the cup" (the hand is full) and "put down the cup on spot B" (the bottle is there), which together held 0.9810 of the Say score at step 2. The Can score does not, however, say why a skill is impossible. The language model sees only the request and the skills chosen so far (Ahn et al. Algorithm 1), so it never learns that the bottle stands on B, and the skill that would help, "pick up the bottle", scores 0.0001 at step 2. Our planner of Section 3 learned the reason as a fact and planned around it.

### 5.4 SayCan on a real robot, and what each method needs

A model as small as GPT-2 is not what SayCan uses. On a mobile robot with one arm, in a mock kitchen built like an office kitchen, SayCan with PaLM, a language model of 540 billion parameters, 4,339 times GPT-2 small, was tested on 101 requests (Ahn et al. §5, Table 2):

- it chose the right skills for 84 percent of them and carried 74 percent out to the end;
- with the language model alone, choosing the skill with the best Say score, 67 percent were planned right;
- on the requests whose answer depends on the scene, 64 percent were planned right with Can and 18 percent without it;
- with the smaller FLAN model, 137 billion parameters, 70 percent were planned right and 61 percent carried out, so PaLM halved the errors (Ahn et al. Table 3).

Most failures on the longest requests came from the language model ending the plan too early, as "done" did on our table (Ahn et al. §5).

| | Task and motion planning (Section 3) | SayCan (Section 5) |
|---|---|---|
| Starts from | a goal written as facts | a request in words |
| Checks the geometry | every action of the whole plan, before the robot moves | the next skill only |
| Learns from a failed test | yes: a new fact | no: the language model never sees why |
| On our table | 4 actions, the cup at B | "pick up the cup", then "done" |
| Needs | a model of every object's shape and place | a language model and skills with affordances |

So the two methods answer the two halves of the opening question: the planner finds actions the arm can carry out, and the language model turns words into skills. Each method has what the other lacks.

## 6. Summary

| Idea | What it does | On the table |
|---|---|---|
| Skill and its numbers | an action with its place, grasp, joint angles and motion, each tested | "pick up the bottle": $x =$ 0.42 m, $q = (31.2, -66.4)$ degrees, 88.7 degrees of motion |
| Plan skeleton | a list of actions with its numbers still open | skeleton 1 fails at B; skeleton 2 moves the bottle first |
| Collision test | two objects overlap if their centres are closer than half their widths added | the cup at B overlaps the bottle by 4 cm, then by 2.49 cm |
| Within-reach test | the hand reaches from 0.1 m to 0.5 m from the shoulder | samples 0.0909 m and 0.0784 m fail |
| Backtracking | back to the last choice when a later test fails | 0.395 m costs 2 motions, then dropped |
| Sequence before satisfy | skeleton first, then numbers; a failed named test becomes a fact | 2 skeletons, 5 samples, 7 motions, 4 kept |
| PDDLStream | streams sample and test values; a PDDL planner searches with them | `s-region`, `s-ik`, `s-motion`, `t-cfree` |
| SayCan | runs the skill with the largest Say times Can, one at a time | GPT-2 small: "pick up the cup", then "done" (1.104 to 1) |

In the table, $x$ is where an object stands and $q$ the arm's two joint angles.

- The cup reaches B in four actions, because our Python planner tests the numbers inside each action: the two-action plan failed with a 4 cm overlap at B, that failure became the fact that the bottle blocks B, and the second skeleton moved the bottle to 0.485 m (after 5 samples) before carrying the cup there, 374.5 degrees of joint motion in all. From the request in words, our SayCan run with GPT-2 small (not SayCan's PaLM) chose "pick up the cup" and then "done" (1.104 to 1), because it scores one skill at a time and never learns that the bottle blocks B.
- A list of actions found from facts alone can fail on the table, because the facts carry no widths or positions; whether one action can run depends on where the earlier actions left things.
- A **skill** (G-3331) hides numbers (place, grasp, joint angles $q$, motion), so its precondition gains tests: on the table, no collision, within reach, a free motion.
- **Task and motion planning** (G-3330) searches for the actions and their numbers together, so it finds plans such as "move the bottle first" that neither a task planner nor a motion planner finds alone.
- A **plan skeleton** (G-3332) is checked by sampling its open numbers, so places that

cannot be listed are still tried, and **backtracking** (G-3334) to the last choice fixes a sample that fails later, at the price of the motions already planned.

- **Sequence before satisfy** (G-3335) wastes no samples on actions no plan needs, while **satisfy before sequence** (G-3336) samples first and searches with tested values; both loop back, so backtracking is part of every order.
- **PDDLStream** (G-3337) writes the samplers and tests as **streams** (G-3333) next to a PDDL domain, so a general PDDL planner can do the action search on real robots; planning takes seconds to minutes and is not changed while the plan runs.
- **SayCan** (G-3338) multiplies the language model's Say score by the skill's **affordance** (G-3339), its chance of succeeding now, because a skill moves a request forward only if it is both the right step and succeeds; so impossible skills are never run, but the choice looks one skill ahead and never learns why a skill is impossible.

The robot can now plan the actions of a task with the arm's geometry in every action, and turn a request in words into its skills when the language model is large enough. One limitation remains: every part of this chapter's stack is designed by hand, from the costmaps to the local planner; the next chapter asks what a local planner learned from experience gains, and at what cost, in "Learned vs classical local planners".

## 7. Sources

### Built from


- underactuated, "Lecture 22: MIT 6.800/6.843 Robotics Manipulation (Fall 2021) |"Task and Motion Planning"", YouTube, https://www.youtube.com/watch?v=A-goUwiIdkA (MIT TAMP lecture; 0:01 to 36:47 and the limits at 54:50 to 55:37 and 75:59)
- Garrett, C. R., Chitnis, R., Holladay, R., Kim, B., Silver, T., Kaelbling, L. P. and Lozano-Pérez, T. (2021). "Integrated Task and Motion Planning". *Annual Review of Control, Robotics, and Autonomous Systems* 4. §1.1 "Example" (the pot, the kettle and the plan skeleton), §2.2 "Multi-modal motion planning", §3.1 "TAMP problem description", §3.3 "Combining action sequence and parameter search" (§3.3.1 sequencing first, Shakey; §3.3.2 satisfaction first; Figure 11), §3.4.2 "Negative methods", Table 1. https://arxiv.org/abs/2010.01083 (Garrett)
- Castro, S. (2022). "Integrated Task and Motion Planning (TAMP) in robotics", Robohub, 16 September 2022: symbolic plans that fail, plan skeletons, sampling a placement pose, backtracking, streams, pyrobosim. https://robohub.org/integrated-task-and-motion-planning-tamp-in-robotics/ (Robohub TAMP)
- Garrett, C. R., *pddlstream*, GitHub repository `caelan/pddlstream`, branch `main`: `README.md` (streams, algorithms, applications) and, in `examples/continuous_tamp`, `domain.pddl` and `stream.pddl`, read 2026-10-12. https://github.com/caelan/pddlstream (PDDLStream README; PDDLStream `continuous_tamp`)
- Ahn, M. et al. (2022). "Do As I Can, Not As I Say: Grounding Language in Robotic Affordances". CoRL 2022. §1, §2, §3 and Algorithm 1 (Say times Can), §5 Tables 2 and 3 (101 instructions), App. D.2 (affordance functions), App. D.3 and Listing 1 (the prompt). https://arxiv.org/abs/2204.01691 (Ahn et al.)

**Other references**

- GPT-2 small weights, `openai-community/gpt2` on huggingface.co, run in NumPy by the repository's `tools/gpt2_small.py`, as in DL-087.

## 8. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

| Term | Meaning |
|---|---|
| Skill (G-3331) | An action a robot can carry out on command, such as "pick up the bottle", with the preconditions it needs and a motion that carries it out; planners and language models build tasks out of skills. |
| Task and motion planning (TAMP) (G-3330) | Planning a list of actions together with the numbers inside every action (where objects go, how they are grasped, the arm's joint angles, the motions) so that each number passes its tests; it finds plans such as "move the bottle off spot B first" that a task planner or a motion planner alone cannot. |
| Plan skeleton (G-3332) | A list of actions whose numbers (places, grasps, joint angles, motions) are still to be found, such as "pick up the bottle, put it down at a new place, pick up the cup, put it down at B"; a planner tests it by searching for numbers that pass every test. |
| Sequence before satisfy (G-3335) | A task and motion planning strategy that first finds a whole plan skeleton from the facts and then searches for its numbers, trying the next skeleton if none pass; it spends no samples on actions no plan needs, but a skeleton can fail late. |
| Backtracking (G-3334) | Going back to the latest choice and trying another value when a later test fails, such as sampling a new place for the bottle when the cup can no longer reach B; it lets a search recover from early choices at the cost of the work done after them. |

| Term | Meaning |
|---|---|
| Satisfy before sequence (G-3336) | A task and motion planning strategy that first samples and tests values (places, joint angles, motions) and then searches for a plan that uses only tested values, sampling more when no plan is found; PDDLStream's incremental algorithm works this way. |
| PDDLStream (G-3337) | An open-source planning framework that extends PDDL with streams, so samplers and tests written in Python sit next to a PDDL domain; its algorithms alternate between sampling values and searching for actions with a PDDL planner, which lets robots plan tasks with geometry. |
| Stream (PDDLStream) (G-3333) | A user-written Python function that returns new values satisfying a condition, such as a place on the table or a motion, or that only tests given values, such as "these two objects do not overlap"; the facts it certifies become preconditions of PDDL actions. |
| Affordance (SayCan) (G-3339) | The probability that a skill, started in the present state, succeeds, such as 0 for "put down the cup on spot B" while a bottle stands there; SayCan multiplies the language model's score by it so impossible skills are never chosen. |
| SayCan (G-3338) | A method that picks a robot's next skill by multiplying a language model's probability that the skill's words come next (Say) by the skill's chance of succeeding now (Can), one skill at a time, so a request in words is turned only into skills the robot can carry out. |
| Nav2 (G-3340) | The navigation stack of ROS 2: separate programs (a lifecycle manager, AMCL, costmaps, a behaviour tree navigator, planner, controller and behaviour servers, a velocity smoother and a collision monitor) that together take a goal pose and drive a robot there. |
| Fact (task planning) (G-2601) | A statement about the task that is either true or false, such as `robot-at table` or `holding-cup`; facts are what a planner's states, actions and goals are made of. |

| Term | Meaning |
|---|---|
| Precondition (G-2606) | The facts that must be true for an action to be done, such as `robot-at table`, `cup-at table` and `hand-empty` for picking up the cup; the planner tries an action only in states that hold all of them. |
| Mode (manipulation) (G-2803) | Which kind of motion applies, transit or transfer, together with where the object stands or how it is held; each mode has its own free space. |
| Probabilistic roadmap (PRM) (G-2780) | A graph of random free configurations (milestones) joined by straight edges that pass a collision check, built once and then searched with A*; it describes the free space without building the obstacle region, so it scales to robots with many joints. |
| Parameter (PDDL action) (G-2614) | A blank in an action, written with a question mark such as `?from` and `?to` in `move ?from ?to`, that the planner fills with objects; one action with parameters replaces many STRIPS actions (6 moves for 3 places, 90 for 10). |
| Collision detection (G-2665) | Deciding, yes or no, whether two bodies overlap, such as the robot's disk and the table; planners call it thousands of times instead of building the obstacle region. |
| Forward search (progression) (G-2617) | Planning by starting at the start state, applying every applicable action to make the next states, and searching the resulting graph of states until one meets the goal; any graph search, such as breadth-first search or A*, can run it. |
| PDDL (Planning Domain Definition Language) (G-2612) | The standard language for writing a planning task, defined in 1998 for the International Planning Competition: actions with parameters, preconditions and effects, in a domain file and a problem file; a task written in it can be handed to any planner that reads PDDL. |

| Term | Meaning |
|---|---|
| Autonomy stack (G-3210) | The whole of a robot's software arranged in layers, sensing at the bottom and mission, behaviour, motion and control above it, each layer running at its own rate and working from the latest output of the layer above; the slow layers plan far ahead while the fast ones react, so a new reading reaches the wheels in one fast cycle. |
| Large language model (LLM) (G-1046) | A transformer language model with billions of parameters, trained on a vast amount of text. |
| GPT (G-855) | A language model that writes text by predicting the next token again and again (Generative Pre-trained Transformer); it is built from transformer decoder blocks only. |
| Log-likelihood (G-1113) | The logarithm of the likelihood, which turns the product of probabilities into a sum of log probabilities; it peaks at the same parameters as the likelihood and is easier to compute and maximise. |
| Reinforcement learning (G-1660) | Learning by acting and receiving rewards or punishments. |